BENTO LABS AI
INDEPENDENT ENGINEERING WORK SAMPLE

BACKEND + FRONTEND

PRODUCT THINKING

CHANGE REVIEW WORKBENCH

A proof-of-work concept for reviewing candidate AI-agent changes with clear evidence, comparable evaluations, and human-controlled release gates.

THE QUESTION THIS CONCEPT EXPLORES

Can an engineer inspect the failure, understand the proposed change, compare its evidence, and record a safe decision from one focused review surface?

CHANGE REVIEW QUEUE

SYNTHETIC CONCEPT DATA

OPEN REVIEWS	AFFECTED CASES	RELEASE GATES
03	54	04
sample queue	seeded cohorts	reviewable checks

01 **TRACE** Failure evidence → 02 **DIFF** Candidate change → 03 **EVAL** Fixture comparison → 04 **DECIDE** Human gate

Built as: FastAPI REST backend + interactive HTML/CSS/JavaScript prototype. The PDF documents the rationale, workflow, implementation and validation plan.

PREPARED BY SRIDHAR

13 OCTOBER 2026

A self-directed concept based on publicly available BentoLabs product information. It is not an official BentoLabs product and does not connect to any company systems.

Ground the idea in the real product

Public company research informs the concept; the proposal does not assume an existing Bento capability is missing.

PUBLIC PRODUCT POSITIONING

“Monitor what runs. Improve what fails. Compound learnings.”

Bento positions itself as production infrastructure for monitoring, debugging and continuously improving long-running AI agents. [1]

01 / Observe

Regression signals, OpenTelemetry-native traces, alerts and behavioral drift help teams surface failure patterns from production runs.

PUBLICLY DESCRIBED

02 / Evaluate

Bento describes evaluating releases against production history, including offline, CI and live-traffic checks.

PUBLICLY DESCRIBED

03 / Compound

Artifacts can represent reusable fixes; The Book records learnings; versions are described as diffable and reversible.

PUBLICLY DESCRIBED

COMPANY CONTEXT

COMPANY	**BentoLabs AI**
FOUNDERS	**Abhinav Soni and Kaushik ASP**
LOCATION / BATCH	**Bengaluru, India · YC Spring 2026**
PUBLIC HIRING SIGNAL	**About page invites builders to reach out via contact@bentolabs.ai**

RESEARCH BOUNDARY

This work sample connects publicly described product primitives into one review-flow concept. It does not claim Bento lacks this workflow, reveal internal product details, or represent knowledge of customer data.

PUBLIC SOURCES REVIEWED

[1] BentoLabs AI product site
https://bentolabs.ai/

[2] BentoLabs AI About / Join Us
https://bentolabs.ai/about

[3] Y Combinator company profile
https://www.ycombinator.com/companies/bentolabs-ai

[4] Bento blog: production agent iteration and regression risk
https://www.bentolabs.ai/blog/vibe-coding-production-agent

Close the review loop with one evidence packet

The target workflow: make a proposed prompt, skill or tool change straightforward to inspect, compare and decide on.

THE JOB TO BE DONE

When a candidate agent change is proposed, help an engineer understand the originating failure, inspect the change, compare it against relevant historical cases, and record a defensible release decision - without hunting across disconnected contexts.

PROPOSED REVIEW JOURNEY

01	02	03	04	05
Failure cohort	**Candidate change**	**Evaluation**	**Release gate**	**Audit record**
Signal + affected runs	Versioned diff	Baseline vs candidate	Critical checks	Owner + rationale
SOURCE	INSPECT	COMPARE	DECIDE	REMEMBER

Primary user: on-call / agent engineer

Has a failure pattern and a proposed fix. Needs enough context to decide whether to keep investigating, rerun evaluation, request changes, or advance the candidate.

Secondary user: reviewer / team lead

Needs consistent evidence, release gates, decision ownership and an audit trail. Should not have to infer risk from a raw diff alone.

DESIGN PRINCIPLES

- **Evidence before opinion**

  Show trace/case references and distinguish observed facts from the root-cause hypothesis.

- **Comparable evaluation**

  Display the baseline, candidate, relevant cohort and explicit regression gates together.

- **Human-controlled change**

  Make promotion reversible and require gates to pass; never silently deploy a generated fix.

- **Inspectable history**

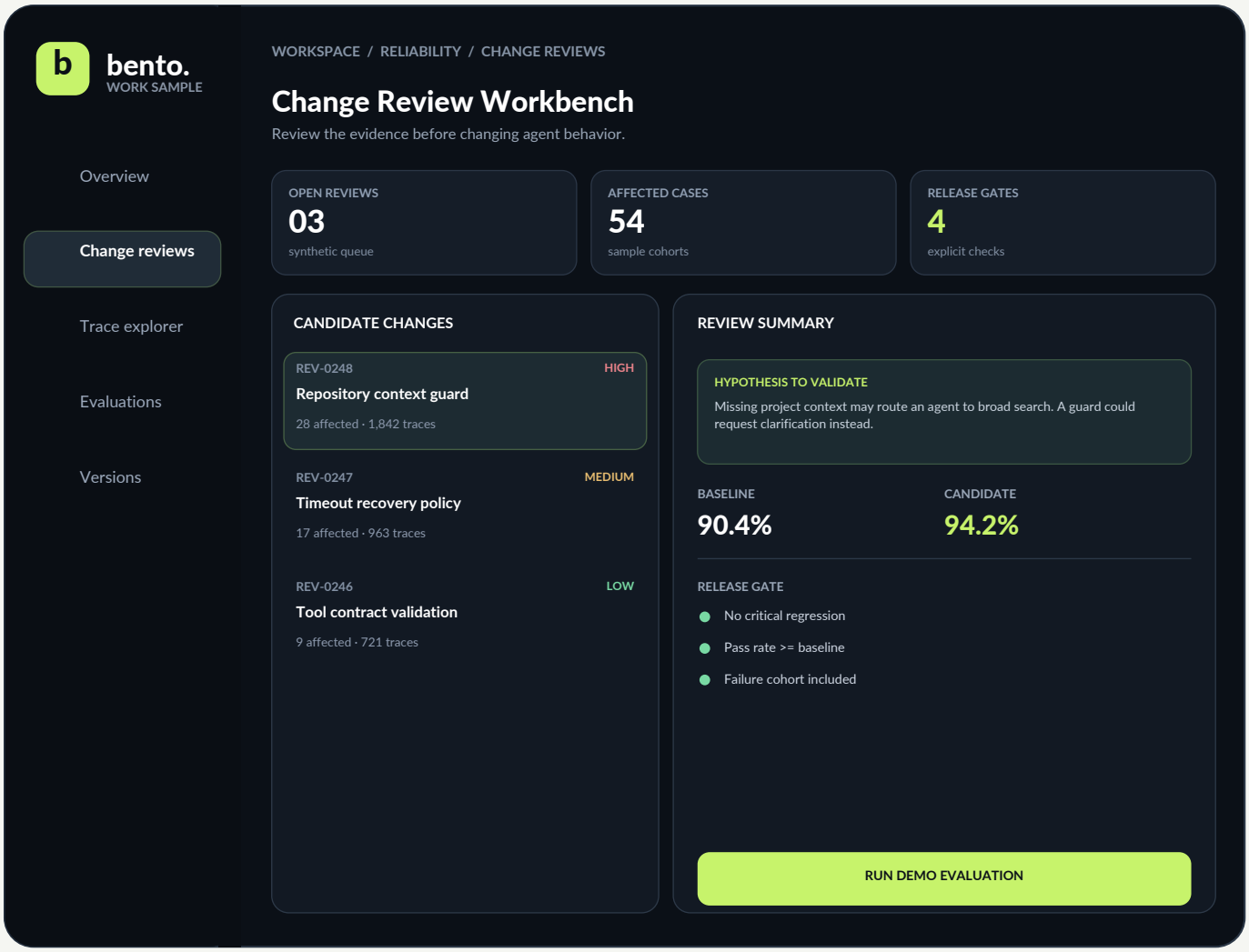
  Record the candidate version, evaluation run, reviewer and decision rationale.

SUCCESS HYPOTHESIS

Reduce the effort needed to assemble review context and make decisions more reproducible. This prototype does not measure a real productivity gain.

A review queue built for fast orientation

The interface makes the next action obvious while keeping the evidence and release gates in view.



A Queue first

Severity, affected cohort and status let the engineer triage before opening a review.

B Side-by-side results

Baseline and candidate sit together, so a percentage is never shown without comparison context.

C Explicit review action

Run an evaluation before making a promotion decision; failed gates block advancement.

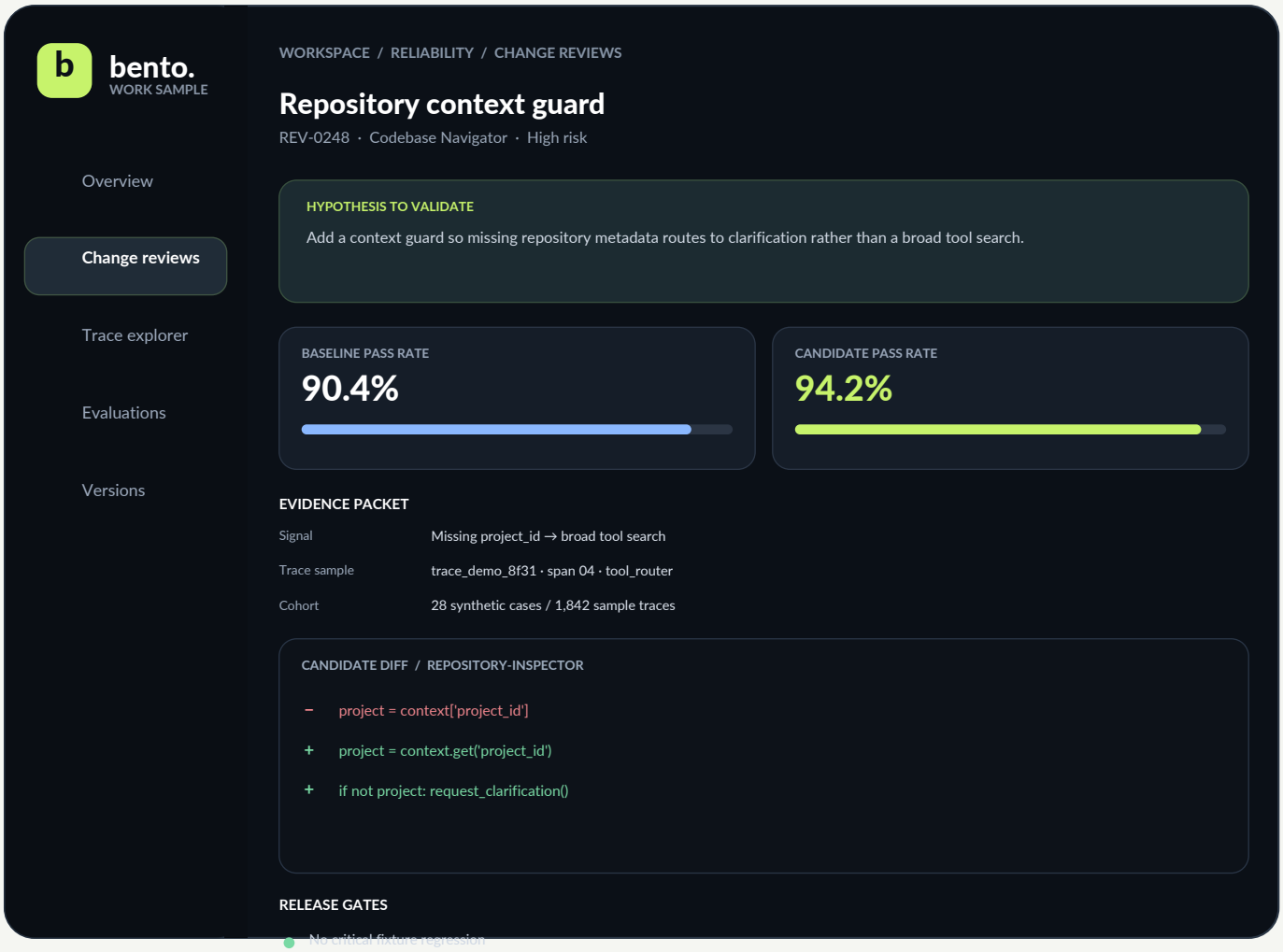
FRONTEND BEHAVIOR IN THE DEMO

Select a candidate; filter by severity; inspect Overview, Evidence, Change diff and Evaluation tabs; run a synthetic evaluation; request changes; or promote only when demo gates pass. The screen uses responsive layout and explicit loading/error feedback.

Important: this UI is an independent concept, not a screenshot of BentoLabs' existing product.

One candidate. Enough context to make a decision.

The packet keeps observed evidence, a proposed fix and the synthetic evaluation clearly separated.



REVIEW PACKET ANATOMY

01 / Observed evidence

Failure signature, trace/span pointer, affected cohort and reproduction fixture.

02 / Proposed change

Readable version diff with a clear scope and a retained rollback target.

03 / Evaluation result

Baseline and candidate pass-rate comparison, latency/cost context, and critical regression count.

04 / Review gate

Named checks, pass/fail reasons, reviewer decision and optional rationale.

DELIBERATE SAFETY CHOICE

Candidate evaluation is not treated as proof of production safety. Promotion is gated on the synthetic checks in this demo; a real system would also require canary strategy, rollback orchestration and team-defined thresholds.

A small full-stack slice, not a slide-only idea

Implemented as a runnable local prototype. Production integrations are described as next steps, not claimed as completed.

REQUEST / DECISION PATH

FRONTEND

HTML / CSS / JS

Tabs, diff, gate state

API LAYER

FastAPI + Pydantic

Typed REST contracts

REVIEW DOMAIN

Candidates / Eval / Decision

Gate checks + audit events

What is implemented

FastAPI REST routes, typed request validation, queue/detail APIs, simulated evaluation, gated decisions, audit events, and an interactive review UI with tabs, filters, and loading/error states.

What is not connected

No BentoLabs credentials, production data, customer traces, LLM provider, telemetry collector, deploy pipeline or real agent runtime. Evaluation fixtures are intentionally synthetic.

REST SURFACE

METHOD	ROUTE	PURPOSE
GET	/api/health	Demo mode + no external connection
GET	/api/reviews	Synthetic review queue
GET	/api/reviews/{id}	Evidence, diff, versions and seeded metrics
POST	/api/reviews/{id}/evaluate	Runs deterministic simulated gate checks
POST	/api/reviews/{id}/decision	Record promote / request-changes / dismiss
GET	/api/audit	Read local in-memory decision/evaluation events

PRODUCTION EVOLUTION IF THE CONCEPT WERE ADOPTED

- **Auth & tenancy**
  RBAC, tenant isolation, secret handling
- **Durable state**
  Database transactions, idempotency, event retention
- **Real evaluations**
  Authorized trace connectors, reproducible fixtures, background jobs
- **Release safety**
  Canary/rollback integration, thresholds, monitoring and audit review

Measure the workflow, not just the UI

The demo tests state transitions and release gates with deterministic fixtures; the proposed product outcomes remain hypotheses.

ILLUSTRATIVE RELEASE GATES

01 **Zero critical regressions**

Block promotion if critical cases fail.

02 **Candidate is not worse**

Compare pass rate to baseline using the same fixture set.

03 **Cohort is represented**

Include the failure pattern that motivated the candidate.

04 **Rollback target exists**

Make the previous artifact/version explicit.

VALIDATION COVERAGE IN THIS WORK SAMPLE

Health and mode contract	Confirms API reports synthetic data + disconnected state
Review listing/detail	Returns stable IDs, evidence, diff and fixture data
Promotion without evaluation	Blocked with HTTP 409
Promotion after passing evaluation	Allowed for demo candidate REV-0248
Critical regression in fixture	Promotion blocked for REV-0247
Frontend workflow	Tabs, filtering, empty/loading/error feedback, decisions

WHAT A REAL RELEASE WOULD STILL NEED

Real historical traces; evaluation reproducibility; severity-weighted gates; statistical confidence; model/prompt/tool version lineage; canary traffic; rollback execution; access control; tenant isolation; audit retention; privacy/security review; and production telemetry. The demo does not claim to validate those properties.

A concrete first slice, ready to inspect

A realistic small scope for a junior full-stack engineer: make the decision path visible, testable and honest about its limitations.

DELIVERY PLAN

PHASE 1	**Understand**	Map the review states, relevant evidence and the assumptions that must be checked with users.
PHASE 2	**Build**	Implement queue/detail UI, evidence packet, diff view and typed FastAPI routes.
PHASE 3	**Validate**	Exercise allowed/blocked transitions; verify API contracts, loading/error states and synthetic-data labels.
PHASE 4	**Review**	Walk through the prototype with an engineer; refine the flow from feedback rather than assuming fit.

WHY THIS IS RELEVANT TO BACKEND + FRONTEND ENGINEERING

Backend: typed REST design, response contracts, validation, release gates, state transitions, error semantics and audit events.
Frontend: a responsive review queue, clear visual hierarchy, evidence panels, data comparison, feedback states and an explicit decision workflow.
Product judgment: a hypothesis tied to a public problem space, reversible decisions, and clear facts vs. assumptions.

LIMITATIONS / HONEST CLAIMS

This is an independent concept and a local prototype. Demo metrics are seeded, not measured. No real AI agent ran; no production trace, secret, or private BentoLabs system was accessed. The UI is not a copy or screenshot of Bento's application. Any overlap with existing capabilities is intentional alignment with publicly described product primitives, not a claim of a product gap.

NEXT STEP

If this workflow is relevant to the team, I would value feedback on one thing: what evidence should be visible before a candidate agent change is considered safe to promote?

RESEARCH LINKS

Bento product | Bento About / Join Us | YC profile | Bento engineering blog

The official About page lists contact@bentolabs.ai for people interested in joining the team. Use the current page to confirm the best route when sending this work sample.